Operating system and system programming (25CS2104E)

Multithreaded Linux Application Using POSIX Threads and Mutexes

Review-I

Under the Esteemed Guidance of

Mr. M. Raghupathi
CSE Department

Team Members:

S Svara Haasini – 2520090170

B Sudharshini – 2520090227

Sumanjali – 2520090190

Contents

Abstract

Modern multi-core systems rely on concurrent execution for efficiency and responsiveness, but when multiple threads within a process access shared resources at the same time, it can cause race conditions and inconsistent results. This is a key challenge in Operating Systems, since threads share the same memory space and, without proper synchronization, can corrupt shared data. This project addresses this by developing a multithreaded Linux application using POSIX threads (Pthreads) for concurrent execution and mutex locks to synchronize access to critical sections, ensuring correctness and thread safety.

Problem Statement

- Modern applications increasingly rely on concurrent execution to use multi-core processors efficiently and stay responsive.
- When several threads within the same process access shared resources simultaneously, it can cause race conditions and data inconsistency.
- Threads share the address space, memory and file descriptors of their parent process — a fundamental OS/Systems Programming challenge.
- Without synchronization mechanisms such as mutexes, concurrent access to shared data can corrupt results and crash the application.
- This project designs and implements a multithreaded Linux application using Pthreads for concurrency and mutex locks for synchronization.

Objectives

- Understand and apply the concept of multithreading in a Linux environment using the POSIX Threads (Pthreads) library.
- Design and implement a C program that creates multiple threads to perform concurrent tasks efficiently.
- Identify critical sections in shared data access and use mutex locks to prevent race conditions and ensure data consistency.
- Evaluate thread synchronization and thread joining, and demonstrate correct, thread-safe program execution on Linux.

Proposed Methodology

- Developed in C on a Linux (Ubuntu) platform using the GCC compiler.
- Main program creates a fixed number of worker threads via pthread_create(), each running a common thread function.
- All threads share the process address space and access a common shared resource (e.g., shared counter or array).
- A pthread_mutex_t protects the critical section — locked before and unlocked immediately after accessing shared data.
- Main thread waits for all workers via pthread_join(), then verifies the final, consistent value of the shared resource.
- Behaviour is tested and compared with and without mutex protection to demonstrate the effect of synchronization.

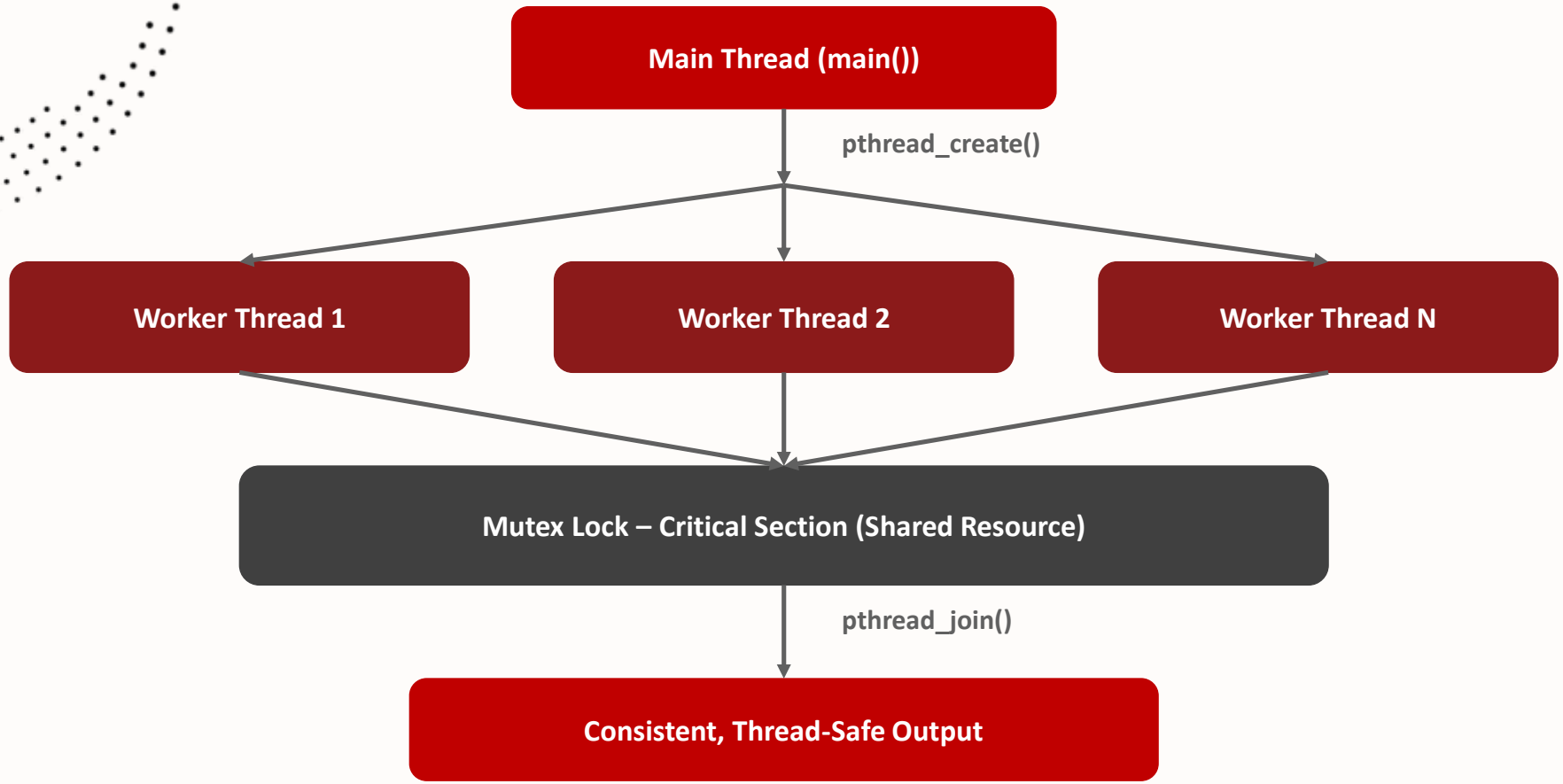
OS Concepts / Linux APIs Used

OS Concept / Linux API	Purpose in the Project
Process & Thread Management	Creating and managing multiple threads within a single Linux process using Pthreads.
pthread_create()	Creates new threads that execute a function concurrently with the main thread.
pthread_join()	Main thread waits for worker threads to finish, ensuring synchronized program flow.
Mutex (pthread_mutex_t)	Implements mutual exclusion so only one thread accesses a critical section at a time.
pthread_mutex_lock()/unlock()	Locks and unlocks the critical section around shared data, preventing race conditions.
Critical Section & Race Condition Handling	Shows how uncontrolled concurrent access corrupts results and how synchronization fixes it.

Tools, Platforms & Software Used

Tool / Platform / Software	Purpose
Linux / Ubuntu	Operating system platform on which the application is developed, compiled and executed.
C / C++	Programming language used for thread creation, mutex synchronization and application logic.
POSIX Threads (Pthreads) Library	Provides the API (pthread_create, pthread_join, pthread_mutex_lock/unlock, etc.).
GCC Compiler	Compiles the C source into an executable, linked with the pthread library (-lpthread).
VS Code / GDB	Used as the code editor and debugger for writing, testing and debugging the application.

System Architecture



Individual Contribution

Roll Number	Student Name	Individual Responsibility
2520090170	S Svara Haasini	Designed the overall program structure, implemented thread creation using pthread_create(), and prepared the project documentation.
2520090227	B Sudharshini	Implemented the mutex locking mechanism (pthread_mutex_lock/unlock) to protect the critical section and tested for race conditions.
2520090190	Sumanjali	Implemented thread joining logic using pthread_join(), verified output consistency, and handled debugging/testing on Linux.

Expected Outcome

- A fully functional multithreaded Linux application that correctly creates, manages and synchronizes multiple POSIX threads.
- Safe concurrent access to shared resources, with mutex locks successfully preventing race conditions.
- Accurate and consistent final output verified across multiple program runs.
- A clear comparison of inconsistent, unpredictable results without synchronization versus consistent, correct results with mutex-based synchronization.
- Practical, hands-on understanding of thread creation, critical sections, mutual exclusion and safe resource sharing in Linux.

Conclusion

- Multithreading improves performance and responsiveness on multi-core systems, but concurrent access to shared data introduces the risk of race conditions.
- Using POSIX threads together with mutex locks effectively synchronizes access to critical sections and guarantees thread safety.
- The project demonstrates core Operating Systems concepts – concurrency, critical sections and mutual exclusion – through a practical Linux implementation.
- It builds a strong foundation for developing more advanced concurrent and parallel systems.

Thank you